

Zanobot · Mess-Labor — der Engine-Benchmark

Umsetzung des Bau-Auftrags „Zanobot · Mess-Labor — Konzept & Bau-Auftrag“ (Stand 27.06.2026) · Bericht erstellt 28.06.2026 · Branch `claude/lese-implementation-009big` · Commit `1a1da41`

Dieses Dokument ist bewusst so geschrieben, dass es von einer anderen KI als Fachgrundlage gelesen werden kann. Es beschreibt *was* gebaut wurde, *warum* und *wie* (auch technisch), und ordnet das Ergebnis in die offene Frage „erst messen, dann bauen“ und die geplante vierte Engine ein.

0 · Kurzfassung

Der im 8-Tage-Statusbericht ausdrücklich offene Punkt — **die Engine-Güte ist noch nie auf Benchmark-Daten gemessen worden** — bekommt jetzt sein Werkzeug. Es wurde ein abgeschottetes, faul geladenes **Mess-Labor** gebaut: ein Desktop-/Experten-Bereich, der einen Ordner mit MIMII-artigen Test-Tönen automatisch durch **alle drei Engines** (GMIA, Spektral-Cosine, YAMNet) schickt und je Maschinen-Bereich je Engine eine **AUC-Note** ausgibt — als Tabelle, exportierbar als CSV/JSON.



Abnahme erfüllt: `tsc --noEmit`, `eslint src`, `npm run build` und `vitest run` sind grün; die GMIA-Regression (25 Tests) ist unverändert grün; der i18n-Konsistenz-Checker meldet weiterhin 1234 Keys × 5 Sprachen vollständig.

1 · Warum — erst messen, dann bauen

Aus dem 8-Tage-Rückblick (§4 „Bewusste Grenzen“ und §7 „Backlog“): Es gibt *keine* echte Feldmessung von AUC/pAUC auf Benchmark-Datensätzen. Ohne diese Zahl ist jede Entscheidung über eine **vierte Engine** ein Bauchgefühl. Das Labor beantwortet zwei Fragen:

- Sind die drei vorhandenen Engines auf realen Daten **gut genug**?
- Wie groß ist die **Lücke** wirklich, die eine vierte Engine schließen müsste?

Das passt exakt zur bewährten Linie des Projekts: *erst messen / schlank / interpretierbar, dann neuronal* (Tier 0 → Tier 1). Das Labor liefert die Baseline, gegen die eine etwaige vierte Engine (nicht-stationäre, bewegte, transiente Geräusche) antreten müsste.

2 · Was die Testdaten enthalten (und wie das Labor sie liest)

Die erwartete Sammlung ist MIMII-artig: tausende kurze WAV-Aufnahmen, sortiert nach Maschine und Zustand. Die Wahrheit steckt allein im Ordernamen:

```
# Kanonische MIMII-Struktur
fan/
  id_00/
    normal/    -> 0000.wav, 0001.wav, ... (gesund)
    abnormal/  -> 0000.wav, ...          (defekt)
  id_02/ ...

# Ebenfalls toleriert
normal/ + abnormal/                (flach -> Maschine "default")
pump/id_00/train/normal/ + .../test/{normal,abnormal}/ (expliziter Split)
```

Faires Messen: Zum Anlernen sieht eine Engine **nur die normal-Aufnahmen** — wie im echten Einsatz, wo nur die gesunde Maschine als Referenz dient. Ein Teil der gesunden Aufnahmen wird zurückgehalten und erst beim Prüfen (gemischt mit den defekten) verwendet, damit sich die Engine nicht selbst benotet.

3 · Architektur-Fit — echter Code, echte Aussage

Kern-Designentscheidung: Das Labor schreibt **keine** Engine-Logik neu. Es ruft dieselben Engine-Instanzen aus der Registry und dieselbe Produktions-Feature-Pipeline auf, die auch im Live-Betrieb laufen. Damit misst es das Produkt, nicht einen Nachbau.

Baustein im Produkt	Verwendung im Labor
<code>registry.getEngine(id)</code>	liefert dieselben Singletons GMIA / Spektral-Cosine / YAMNet
<code>extractFeatures(buffer, config)</code>	330-ms-Frames, 512-Bin relative ESD — unverändert
<code>DiagnosisEngine.train / classify</code>	Anlernen & Scoring je Frame (synchron)
<code>AsyncDiagnosisEngine (YAMNet)</code>	1:1 genutzt: lazy <code>init()</code> , 16-kHz-Resample, ~1-s-Fenster
<code>FrameInput (feature + rawChunk)</code>	Sync-Engines nehmen das Feature, YAMNet den Roh-Chunk

„GMIA ist heilig“: Default-Pfad, $\lambda = 1e9$ und die Echtzeitschleife `processChunkDirectly()` wurden nicht angefasst. Das Labor ist ein eigener, faul geladener Modul-Block (`src/lab/`), aus dem normalen Flow nicht erreichbar.

4 · Was das Labor Schritt für Schritt tut

- Ordner einlesen.** File System Access API (`showDirectoryPicker`), Fallback `<input webkitdirectory>`. Die `normal/abnormal`-Struktur wird selbst erkannt.
- Split planen.** Expliziter `train/test` wird verwendet; sonst werden die normal-Dateien **deterministisch** (nach Name sortiert, Default 50 %/50 %) geteilt. Der gewählte Split wird im Zeugnis ausgewiesen.
- Anlernen.** Aus den gesunden Trainings-Clips baut jede Engine ihr Referenzmodell — mit demselben echten Code wie im Produkt. (In-Memory; nichts wird persistiert.)
- Durchspielen.** Jeder Prüf-Clip (gesund & defekt) läuft durch jede Engine; pro Frame entsteht ein Health-Score, daraus eine Auffälligkeit (`100 - score`).
- Aggregieren.** Die Frame-Auffälligkeiten werden zu **einem** Clip-Wert zusammengefasst (`CLIP_AGG = 'mean' | 'p90'`, Default `mean`).

6. **Benoten.** Je Bereich je Engine wird die AUC berechnet (`normal=0`, `abnormal=1`), zusätzlich pAUC bei $FPR \leq 0,1$.
7. **Zeugnis zeigen.** Tabelle Maschinentyp $\times$ Engine inkl. Zeile „Mittel“; Download als CSV und JSON. Nichts landet in echten Maschinen-Referenzen.

5 · Die Note (AUC) — Definition & Implementierung

Eine Engine gibt keinen Stempel „gesund/krank“, sondern eine Zahl. Statt sich auf eine willkürliche Grenze festzulegen, fasst die AUC die Trennschärfe über *alle* Grenzen in einer Zahl zusammen:

$$AUC = P(\text{Score}(\text{zufällig defekt}) > \text{Score}(\text{zufällig gesund}))$$

AUC	Bedeutung
1,0	perfekt — jede defekte Aufnahme auffälliger als jede gesunde
0,85	richtig gut
0,70	brauchbar, aber wackelig
0,50	wertlos — die Engine rät praktisch

Implementiert ist die AUC **rangbasiert** (Mann-Whitney-U), wodurch Gleichstände exakt über mittlere Ränge behandelt werden — robuster als ein Schwellen-Sweep. Die pAUC nutzt einen ROC-Sweep und wird auf den FPR-Bereich $[0, 0,1]$ normiert (operativ zählt nur der Bereich niedriger Fehlalarme).

5.1 · Reine, unit-getestete Funktionen

Die mathematischen Kerne sind dependency- und browser-frei und damit isoliert testbar — genau der Vertrag des Bau-Auftrags:

Funktion	Signatur	Zweck
<code>auc</code>	<code>(labels, scores) → [0,1] NaN</code>	Trennschärfe, Ties-sicher; NaN wenn eine Klasse fehlt
<code>pAUC</code>	<code>(labels, scores, maxFpr=0.1)</code>	partielle AUC bei niedrigem FPR, normiert
<code>clipAggregate</code>	<code>(frameScores, 'mean' 'p90')</code>	viele Frames → 1 Clip-Wert; p90 = „schlechtestes Zehntel“
<code>parseFolder</code>	<code>(paths[]) → ParsedDataset</code>	Ordnerstruktur → Bereiche mit normal/abnormal
<code>planSplit</code>	<code>(section, ratio=0.5) → SplitPlan</code>	train/test verwenden oder deterministisch teilen

6 · So sieht das Zeugnis aus

Die Ergebnis-Ansicht ist die Tabelle Maschinentyp $\times$ Engine mit einer „Mittel“-Zeile. Die folgenden Zahlen sind **Beispielwerte zur Veranschaulichung** — die echten Zahlen liefert erst die Messung auf einem Gerät mit dem entpackten Datensatz.

Maschinentyp	GMIA	Spektral-Cosine	YAMNet	Split
Lüfter (fan)	0,91	0,85	0,79	auto 50 %
Pumpe (pump)	0,88	0,90	0,71	auto 50 %
Ventil (valve)	0,76	0,85	0,77	train/test
Mittel	0,85	0,87	0,76	

Fiktive Werte. Genau diese Tabelle ist das Ziel; CSV/JSON-Export enthält zusätzlich pAUC, Clip-Zahlen je Klasse und etwaige Engine-Fehler je Bereich.

7 · Stolperfallen — vorab bedacht & umgesetzt

Risiko	Umsetzung im Labor
YAMNet ist das langsamste Glied	Fortschrittsanzeige (X/N), Abbrechen-Knopf (AbortSignal); Modell nur einmal lazy geladen
Tab friert ein	kooperatives Yield (<code>requestAnimationFrame</code>) nach jedem Clip; Sync-Arbeit in Häppchen
Große Ordner	optionale Obergrenze „Max. Dateien/Bereich“ für die erste Messung (z. B. nur <code>id_00</code>)
Gleiche Tonqualität	identische Abtastraten-/Resampling-Logik wie im Produkt; <code>decodeAudioData</code> + Produktions-Config je Datei
Pro Aufnahme ein Wert	Frame-Auffälligkeiten → mean oder p90 („schlechtestes Zehntel“)
Ehrliche Aufteilung	gesunde Clips in Anlern-/Prüf-Teil getrennt; Split (Quelle + Verhältnis) im Zeugnis dokumentiert

8 · Isolation, Bundle & Erreichbarkeit

- **Code-Split / lazy:** Das gesamte Labor ist ein eigener ~21-kB-Chunk (8 kB gzip), nur über `import('@lab/index.js')` erreichbar. Das Hauptbundle bleibt praktisch unverändert; TF.js bleibt sein eigener 1,9-MB-Chunk, der *nur* bei YAMNet-Wahl lädt.
- **Desktop-/Experten-Gate:** Der Eintrag erscheint nur, wenn `showDirectoryPicker` vorhanden *und* die Ansicht breit genug ist (≥ 900 px) — Ordner-Einlesen und hunderte YAMNet-Läufe sind nichts fürs Handy. Einstieg über Einstellungen → Analysemethode (Experten-Ebene).
- **Lokale Texte:** Die Labor-Oberfläche nutzt eigene deutsche Strings statt i18n-Schlüssel — so bleiben die fünf Sprachsätze und der Konsistenz-Checker unangetastet.

9 · Dateien & Tests

Datei	Inhalt	Tests
src/lab/auc.ts	auc , pAUC	12
src/lab/clipAggregate.ts	clipAggregate (mean/p90)	7
src/lab/parseFolder.ts	parseFolder , planSplit	8
src/lab/benchmark.ts	Runner: Decode, Train, Score, Fortschritt, Abbruch	— (Engines/Browser)
src/lab/MessLaborView.ts	Overlay-UI, Ordnerwahl, Tabelle, Export	—
src/lab/exportResult.ts · types.ts · index.ts	CSV/JSON, Typen, Lazy-Einstieg	—

Bestandsänderungen rein additiv (42 Zeilen): `index.html` (Eintrags-Button), `4-Settings.ts` (Lazy-Wiring, nur Desktop), `@lab` -Alias in `tsconfig/vite/vitest`.

10 · Grenzen dieser Umsetzung (ehrlich)

- Die **echten Zahlen fehlen noch**: Der Runner und die UI laufen nur im Browser auf einem Gerät mit dem entpackten Datensatz. Die hier gezeigten AUC-Werte sind fiktiv.
- Die **Inferenz** (TF.js-Lauf von YAMNet) ist nur auf einem echten Gerät validierbar; die umgebende Struktur (Resampling, Fensterung, Aggregation, AUC) ist unit-getestet.
- YAMNet wird im Labor über ~1-s-Fenster über das Roh-Audio bewertet (analog zum 1-s-Ringpuffer im Produkt). Das ist die faire, produktnahe Granularität — aber YAMNets 0,96-s-Fenster bleibt grob für kurze Transienten (bekannte Beta-Grenze).

11 · Anschluss an die Diskussion zur 4. Engine

Das Labor ist der im Rückblick §6.6 geforderte Schritt „erst messen“: Es liefert die **Baseline** der drei stationär gedachten Engines auf instationären/getakteten Fällen (MIMII DG, später ToyADMOS2) und beziffert damit, *wie groß* die Lücke wirklich ist. Konkrete Fragen, die die Messung jetzt beantwortbar macht und die die andere KI mitbewerten kann:

1. Wie stark fällt YAMNet gegenüber GMIA/Spektral-Cosine bei **p90**-Aggregation ab — also wenn kurze Transienten den Ausschlag geben? (Indikator für den Mehrwert eines Sequenz-/Ereignis-Modells.)
2. Auf welchen Maschinentypen ist die AUC bereits hoch ($\geq 0,9$) — dort lohnt eine vierte Engine nicht — und wo bleibt sie wackelig ($\leq 0,75$)?
3. Ist ein trainingsfreier **DTW/Matrix-Profile**-Einstieg (Tier-2-trivial) gerechtfertigt, bevor ein neuronales Sequenzmodell kommt — gemessen an der Lücke, die das Labor zeigt?
4. Welche Benchmark-Teilmengen isolieren instationäre/bewegte/transiente Fälle am saubersten, damit die Messung die richtige Lücke quantifiziert (nicht nur den stationären Durchschnitt)?

Methodisch ist das Labor bewusst *additiv* und engine-agnostisch: eine vierte Engine (`engineId: 'temporal'` o. ä.) ließe sich später ohne Änderung am Labor mitmessen, sobald sie das bestehende `DiagnosisEngine / AsyncDiagnosisEngine` -Interface erfüllt.

`src/core/dsp/features.ts` . Beispiel-AUC-Werte sind fiktiv; die echten Zahlen liefert die Messung.